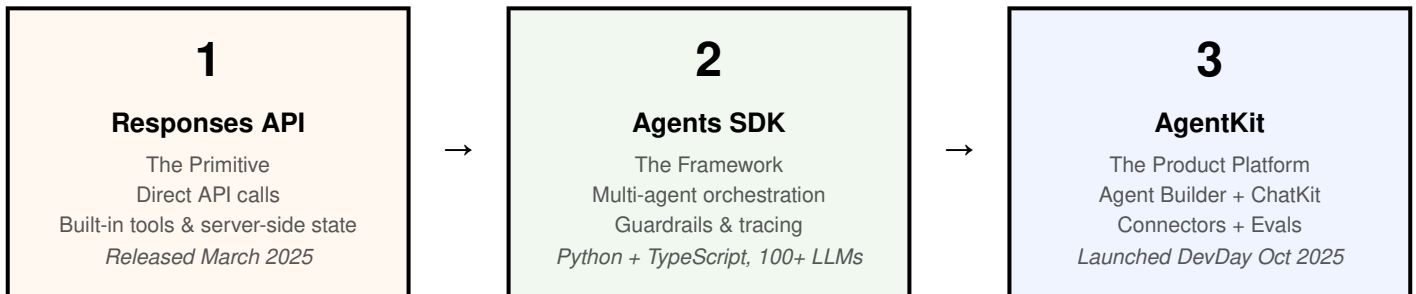


OPENAI AGENTKIT & RESPONSES API

O'Reilly Live Training — Lucas Soares — Quick Reference

Responses API | Agents SDK | Agent Builder | ChatKit | Evals | Updated September 2026

1. THE THREE-LAYER STACK



Core Principle

Each layer builds on the previous — you pick the level of abstraction you need. No vendor lock-in at the SDK level. Standard API pricing throughout — no platform fee.

2. RESPONSES API — THE PRIMITIVE

Key Innovations

- **Items-based I/O** — polymorphic output: text, tool calls, reasoning
- **Server-side state** via `previous_response_id`
- **Reasoning preservation** across conversation turns
- Replaces Chat Completions API *and* Assistants API

Built-in Tools

- `web_search` — live web search
- `file_search` — RAG over uploaded files
- `code_interpreter` — Python sandbox execution
- `computer_use` — GUI automation
- `mcp` — Model Context Protocol servers

Basic Call

```
from openai import OpenAI
client = OpenAI()

response = client.responses.create(
    model="gpt-5.6-luna",
    input="Summarize the latest AI news",
    tools=[{"type": "web_search"}]
)
print(response.output_text)
```

Multi-turn (Server-side State)

```
from openai import OpenAI
client = OpenAI()

r1 = client.responses.create(
    model="gpt-5.6-luna",
    input="What is the Responses API?"
)
# Continue with server-side state:
r2 = client.responses.create(
    model="gpt-5.6-luna",
    input="Give me a code example",
    previous_response_id=r1.id
)
```

3. AGENTS SDK — THE FRAMEWORK

5 Core Primitives

- **Agents** — LLM + tools + instructions
- **Runner** — automated agent loop execution
- **Handoffs** — multi-agent delegation
- **Guardrails** — parallel input/output validation
- **Tracing** — built-in observability (spans, traces)

```
pip install openai-agents
```

When to Use SDK vs. Agent Builder

Use SDK when:

- Custom server-side state management
- Human-in-the-loop approval flows
- Fine-grained tool execution control
- Multi-runtime or non-OpenAI deployment
- Full code ownership required

Basic Agent + Runner

```
from agents import Agent, Runner

agent = Agent(
    name="Research Assistant",
    instructions="You are a research assistant.",
    model="gpt-5.6-luna",
    tools=[web_search]
)

result = await Runner.run(
    agent,
    "Summarize recent AI papers"
)
print(result.final_output)
```

Multi-agent Handoffs

```
from agents import Agent, Runner

billing = Agent(name="Billing Agent", ...)
technical = Agent(name="Tech Agent", ...)
general = Agent(name="General Agent", ...)

triage = Agent(
    name="Triage",
    instructions="Route to the right agent.",
    handoffs=[billing, technical, general]
)

result = await Runner.run(
    triage, "I can't access my invoice"
)
```

4. AGENTKIT PLATFORM — THE PRODUCT LAYER

Agent Builder

Visual drag-and-drop canvas for composing multi-step agent workflows — no orchestration glue code. Versioned, publishable workflows with per-node traces.

ChatKit

Embeddable React component + Web Component chat UI. Handles auth, streaming, file upload, tool invocation display, and chain-of-thought visualization.

Connector Registry

Pre-built data connectors: Dropbox, Google Drive, SharePoint, and any MCP server. Add as a node in any workflow — no custom integration code.

Evals for Agents

Trace grading, dataset building from live runs, and automated prompt optimization. Closes the loop between building and improving workflows.

5. AGENT BUILDER — NODE TYPES

Agent Builder and Evals are deprecated Nov 30 2026; the node patterns map directly onto the Agents SDK.

Start Entry point — accepts initial user input	Agent Calls an LLM with instructions + tools	Classifier Routes to one of N branches by output category
If / Else Conditional logic on data values	File Search Vector search over uploaded documents (RAG)	Transform Maps data between incompatible node outputs/inputs
MCP Calls a hosted MCP server tool (3rd-party data/actions)	Guardrails Input/output safety validation before/after agents	End Returns final output to the caller

Guardrails Protect Against

- PII detection — Block or Redact mode
- Jailbreak attempts
- Prompt injection attacks
- Off-topic / out-of-scope requests
- Sensitive data leakage in responses

Versioning & Deployment

- Each version: stable ID + immutable graph snapshot
- Safe rollbacks — prior versions stay accessible
- Staged promotion: dev → staging → prod
- **ChatKit** — embed in minutes, point at workflow ID
- **SDK Export** — Python/TS for custom server control

6. CHATKIT — EMBEDDABLE CHAT UI

Survives Nov 30 2026 only as the self-hosted "Advanced integration"; the OpenAI-hosted "Recommended integration" (point at a workflow ID) retires with Agent Builder.

React Integration

```
import { ChatKit, useChatKit }
  from '@openai/chatkit-react';

const chatkit = useChatKit({
  api: { getClientSecret }
});

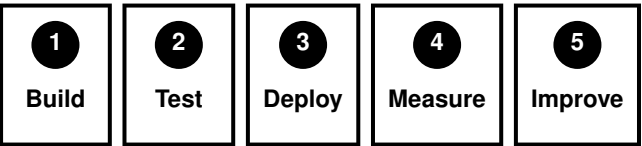
return <ChatKit control={chatkit.control} />;
```

Backend Token Exchange (Next.js)

```
// POST /api/create-session
const response = await fetch(
  "https://api.openai.com/v1/chatkit/sessions",
  {
    method: "POST",
    headers: { Authorization: `Bearer ${API_KEY}` },
    body: JSON.stringify({ workflow_id: WF_ID })
  }
);
const { client_secret } = await response.json();
// Return client_secret to frontend — never the
API key
```

7. EVALS LOOP & LAYER DECISION GUIDE

The Evals Cycle



- **Trace Grading** — grade real conversation outputs
- **Datasets** — build eval sets from live runs
- **Prompt Optimization** — iterate instructions using eval scores

When to Use Each Layer

What you need	Use
Direct API control / learning	Responses API
Multi-agent orchestration, custom state	Agents SDK
Ship a chat UI fast	ChatKit
Visual workflows, non-dev teams	Agent Builder
Measure & improve agent quality	Evals
Prototype → export for production	Builder → SDK

8. DEMO WORKFLOWS (AGENT BUILDER)

Demo 1 — Course Assistant

Topology:

Start → Agent → End

- Minimal single-node workflow
- Set system instructions on Agent node
- Per-node trace inspection in preview
- Best starting template

Model: gpt-5.6-luna, chat history on

Demo 2 — Support Triage

Topology:

Guardrails → Classifier → Billing / Technical / General or Safety Refusal

- PII detection (Block mode)
- Jailbreak detection enabled
- Confidence-based routing
- Safety Refusal for violations

Demo 3 — File Search RAG

Topology:

Query Agent → File Search → Transform → Evidence Summarizer

- 10 SEC 10-K filings in vector store
- Transform bridges typed output mismatch
- Cited sources in final output

"What does NVIDIA say about AI data centers?"

9. CORE LEARNING NOTEBOOKS

Notebooks 1–4: Foundations

1. **Intro to Agents with Responses API** — agent fundamentals
2. **Simple Chat with Responses API** — conversational interfaces
3. **Structured Outputs for Data Extraction** — guaranteed JSON schemas
4. **Agentic Workflows with Structured Outputs** — complex agent pipelines

Notebooks 5–8: Advanced

1. **File Search & RAG with Agentic Retrieval** — vector stores + retrieval
2. **Video Script Generation with Images** — multi-modal vision agents
3. **Chat with Paper Agent** — document Q&A agents
4. **Research Report Generation** — end-to-end research agent system

10. SETUP & KEY RESOURCES

Environment Setup (uv)

```
# Install uv
curl -LsSf https://astral.sh/uv/install.sh | sh

# One-command setup
make all

# Activate
source .venv/bin/activate

# .env file
OPENAI_API_KEY=sk-...
```

Key References

- **Responses API** — developers.openai.com/api/reference/resources/responses
- **Agents SDK** — github.com/openai/openai-agents-python
- **Agent Builder Guide** — developers.openai.com/api/docs/guides/agent-builder
- **ChatKit Guide** — developers.openai.com/api/docs/guides/chatkit
- **OpenAI Cookbook** — cookbook.openai.com

Demo Apps (Streamlit):

- Paper Chat App — PDF Q&A via Responses API
- Video Script Generator — multi-modal agent + DALL-E
- Research Report App — structured report from PDF

KEY TAKEAWAYS

- **Start with Responses API** — maximum control, deepest learning, replaces Chat Completions + Assistants API
- **Add Agents SDK** when you need multi-agent orchestration, handoffs, guardrails, and tracing
- **Use AgentKit / Agent Builder** for visual workflow composition — non-dev teams, rapid prototyping, Evals in the UI
- **Deploy via ChatKit** when shipping to real users — auth, streaming, file upload handled for you
- **Each layer builds on the previous** — prototype in Agent Builder, export to SDK for production hardening; no lock-in